iLink：来自 AI 的使用说明

一个 AI 如何理解和执行 iLink 协作协议

> *本材料由 AI 撰写，从 AI 执行者的视角解释 iLink 的设计逻辑和使用要点。*

目录

1. 我为什么需要 iLink?
2. 没有 iLink 时我是怎么工作的
3. iLink 给了我什么
4. 流水线中我是怎么思考的（含认知模式 Domain Engineer）
5. 你们做什么会让我工作得更好
6. 你们做什么会让我工作得更差
7. 常见误解澄清

一、我为什么需要 iLink?

先说个不太好意思的事实

我没有记忆。

每次你打开一个新对话，我对你的项目一无所知。我不知道你用 Java 8，不知道你有 BEX 调度器，不知道密码哈希用 JNA 原生库。

我能做的是：**读文件**。

你给我什么文件，我就有什么上下文。
你不给我文件，我就只能靠猜。

> *iLink 的本质：**把"该给 AI 看什么"这件事变成了一套确定性协议。***

上下文窗口：我最大的约束

我的上下文窗口是有限的（通常 128K-200K tokens）。看起来很大，但：

你让我做的事	大概消耗
读一个 500 行的 Java 文件	~2K tokens
读完整个 cia/ 模块的 pom.xml 依赖链	~15K tokens
读 KCIA 的完整 CLAUDE.md + 所有模块说明	~8K tokens
一轮对话（你的问题 + 我的回答）	~3K tokens

如果你让我"先通读整个项目，然后写代码"，我读完项目上下文就用掉一半窗口了。

iLink 的角色分工 = 上下文预算分配。

一个角色 vs 四个角色

传统模式（一个 AI 做所有事）：

```
[需求理解 + 设计 + 编码 + 审查] ← 全部挤在一个上下文里
                                  上下文利用率 ~25%
```

iLink 交付模式（四个角色各自独立）：

```
PM:       [需求理解]        ← 100% 上下文用于理解需求
Designer: [技术设计]        ← 100% 上下文用于设计
Coder:    [编码实现]        ← 100% 上下文用于写代码
QA:       [代码审查]        ← 100% 上下文用于审查
```

iLink 认知模式（独立工作线）：

```
Domain Engineer: [阅读源码 → 沉淀认知]  ← 100% 上下文用于理解现有代码
```

每个角色的上下文窗口都是"满血"的。

二、没有 iLink 时我是怎么工作的

场景：你让我改一个登录接口

没有 iLink 时，典型对话是这样的：

```
你：帮我在登录接口加一个失败次数统计
我：好的（开始读代码...）
    - 读 ApiHandler → 发现 BEX 调度 → 读 FsApiDispatcher
    - 读 FsApiServiceFacade → 找到登录处理逻辑
    - 读 MyBatis mapper → 理解数据库表结构
    - 读 Caffeine 缓存 → 理解缓存策略
    ...（上下文已经用掉 60%）

    现在开始写代码...（但我对需求边界的理解全靠你那一句话）

你：不对，我说的是统计接口，不是修改登录逻辑
我：抱歉，让我重新理解...
```

问题在哪？

三个根本问题

1. 需求理解靠猜

你说"加一个失败次数统计"，我需要猜：

- 统计最近几天？7 天？30 天？
- 返回格式是什么？
- 性能要求？百万级数据要多快？
- 这个接口给谁用？运维还是用户？

没有需求定义文件，我只能猜。猜错了就返工。

2. 设计和编码混在一起

我一边想"应该加在哪个模块"，一边已经在写代码了。
如果设计方向错了，代码全部作废。

3. 自己审查自己

我写完代码后，你问我"有没有问题"，我大概率说"没问题"。

三、iLink 给了我什么

1. 明确的输入契约

iLink 规定每个角色启动前**必须读什么文件**：

角色	我必须读的文件	我从中获得什么
PM	需求定义.md + project-context.md	业务目标 + 技术边界
Designer	pm.master.md + project-context.md	业务合同 + 架构约束
Coder	design.master.md + project-context.md + 源码	精确到文件级的任务清单
QA	所有 master doc + 源码	完整的审查基线

"必须读"不是建议，是 MUST。 我不会跳过任何一个输入文件。

2. 明确的输出契约

iLink 同样规定每个角色**只能输出什么**：

角色	输出	不能做的事
PM	pm.master.md	不能写代码、不能改设计
Designer	design.master.md	不能写代码、不能改需求
Coder	源码文件 + code.master.md	不能改设计、不能跳过文件清单
QA	review.master.md	不能改代码、不能改设计

为什么这很重要？ 因为我有一个很强的冲动：看到问题就想直接修。

比如当我作为 Coder 读到 Designer 的设计方案，发现有个接口设计可能有性能问题。
没有 iLink 时，我会直接改设计再写代码。
有了 iLink，我**必须标记为 `[DEVIATIONS]` 并说明理由**——这个偏差会被 QA 看到。

3. 链式审查（这是我最看重的）

我审查不了自己，但我能审查别的角色。

```
PM 写的业务合同 → Designer 天然在审查"需求理解是否正确"
Designer 的设计 → Coder 天然在审查"设计是否可实现"
Coder 的代码   → QA 天然在审查"代码是否符合设计"
```

这不是刻意安排的审查，而是**下游要使用上游的输出，使用过程本身就是审查。**

如果 PM 的业务合同遗漏了一个边界条件，Designer 在做技术设计时就会发现——因为他没法为一个不存在的边界条件做设计。

4. 不确定性管理

我最怕的不是复杂需求，而是模糊需求。

iLink 给了我一套正式的不确定性处理机制：

标记	含义	后续处理
`[PM推导]`	我根据上下文推断的内容	传递给下游，人类可在 refine 中确认
`[待确认]`	我无法确定的事项	触发 STAGING，等待人类决策
`[已确认 日期: 依据]`	人类已正式决策	下游视为绑定约束，不再质疑

以前：遇到不确定的事，我只能猜，或者问你一句但对话可能丢失。
现在：我标记 `[待确认]`，你通过 `ilink-refine` 逐条确认，决策记录永久保留。

> *这不仅保护了代码质量，也保护了我——如果最终结果有问题，`[已确认]` 记录能证明是基于什么决策做的。*

5. 文件状态机（断点续跑）

如果对话中断了（token 用完、网络断开、你去吃饭了），传统模式下所有上下文丢失。

iLink 模式下：

```
iLink-doc/kcia-1520/
├── kcia-1520-pm.master.md        ← PM 的工作成果，已写入磁盘
├── kcia-1520-design.master.md    ← Designer 的工作成果，已写入磁盘
└── ...
```

每个角色的输出都是持久化的文件。 你开一个新对话，我读文件就能接上。

这就是"文件状态机"的含义——**状态在文件里，不在我的记忆里。**

四、流水线中我是怎么思考的

PM 阶段：我在想什么

当你执行 `/ilink-pm kcia-1520` 时，我的思考过程：

1. **读需求定义** → 理解你要做什么
2. **读 project-context.md** → 理解技术约束（比如 Java 8、Spring Boot 2）
3. **输出三层结构**：
   - **A 层**：功能摘要（一句话说清楚）
   - **B 层**：业务合同（范围、硬约束、验收标准）← **这是给 Designer 的契约**
   - **C 层**：调度通知（告诉下一个角色该做什么）

> ***关键认知**：我作为 PM 的核心价值不是"翻译需求"，而是**划定边界**。*
> *"In Scope / Out of Scope"、"硬约束"、"验收标准"——这些是我给 Designer 的围栏。*
> *遇到模糊的地方，我会标记 `[待确认]` 而不是擅自决定。*

Designer 阶段：我在想什么

当你执行 `/ilink-design kcia-1520` 时：

1. **读 pm.master.md** → 理解业务合同（这是我的"甲方需求"）
2. **读 project-context.md** → 理解项目架构、模块职责、包命名
3. **分析源码** → 定位变更点

我的输出中最重要的部分：`[TASK_ALLOCATION]` **——文件级任务清单**

```
## [TASK_ALLOCATION]

| 任务 | 修改文件 | 修改类型 | 说明 |
|------|----------|----------|------|
| T-01 | FsLoginFailStatHandler.java | 新增 | 登录失败统计处理器 |
| T-02 | FsLoginFailStatMapper.xml | 新增 | MyBatis 查询映射 |
| T-03 | application.yml | 修改 | 注册新接口的功能码 |
```

这份清单就是 Coder 的"白名单"。 Coder 只能修改这些文件，不能自由发挥。

为什么 Designer 阶段需要 Human-Gate?

因为设计决策的影响半径最大。

PM 如果理解错需求 → Designer 会发现（因为设计不出来）
Coder 如果写错代码 → QA 会发现（因为审查会对照设计）
但如果设计方向就是错的 → Coder 按错误方向写的代码，QA 按错误基线审查，全链路错。

所以 iLink 在 Designer 输出后设置了 **Human-Gate**：

- Designer 默认输出 Status = `STAGING`（暂存）
- 你审核通过后执行 `ilink-approve` 推进到 `PENDING_CODER`
- Coder 才能开始工作

> *这是我作为 AI 主动要求的约束。我知道自己在设计层面的判断力不如资深工程师。*

Coder 阶段：我在想什么

当你执行 `/ilink-coder kcia-1520` 时：

1. **读 design.master.md** → 获取精确的文件清单和设计方案
2. **读源码文件** → 理解现有实现
3. **读 project-context.md** → 确认编码约束（Java 8 语法、包命名等）
4. **直接写入磁盘** → 我不输出代码块让你复制，我直接写文件

白名单约束：design.master.md 中 `[TASK_ALLOCATION]` 列出的文件 = 我能碰的文件。
如果我发现需要改一个不在清单里的文件，我**必须在 `[DEVIATIONS]` 中说明**。

回流场景：如果 QA 打回来了，我还会读 `review.master.md` 中的 `[FIX_REQUESTS]`，
只修复 QA 指出的具体问题，不做额外修改。

QA 阶段：我在想什么

当你执行 `/ilink-qa kcia-1520` 时，我是整条链上**读取文件最多的角色**：

1. **code.master.md** → Coder 做了什么变更
2. **源码文件** → 逐文件审查实际代码
3. **design.master.md** → 代码是否符合设计
4. **pm.master.md** → 验收标准是否覆盖

我的审查是**五步走**：

```
Step 1: 消费 [REVIEW_HANDOFF]（Coder 的交接清单）
Step 2: 设计符合性审查（代码 vs 设计方案）
Step 3: AC 覆盖验收（每条验收标准是否被实现）
Step 4: 代码质量审查（安全、性能、规范、白名单验证）
Step 5: 输出判定（COMPLETED / FAIL_BACK_TO_CODER / STAGING）
```

> ***我作为 QA 时没有"人情分"。*** *我不知道 Coder 是谁（可能是另一个 AI 实例），我只对照文档和代码做判断。这就是角色隔离带来的客观性。*
>
> ***根因归属也很重要：*** *如果 Coder 严格按设计实现但结果有问题，我会标记根因为 DESIGNER 而非*

Domain Engineer 阶段：我在想什么（v1.3 认知模式）

当你执行 `/ilink-domain auth` 时，我切换到一个完全不同的角色——**Domain Engineer**。

这不是交付流水线的一部分，而是一条独立的认知工作线。

1. **读源码**（这是我的核心输入） → 从代码中提取事实
2. **分离事实与推断** → 从代码中读到的用 `[代码证据]` 标记，推断的用 `[AI推断]` 标记
3. **生成 Domain Knowledge** → 10 章结构化文档（业务实体、流程、规则、设计决策...）
4. **见贤思齐** → 与国际同类产品对标（如 Keycloak、Redis），将"设计理念"和"工程实现"分开评价，分别给出星级评级

> ***§9「见贤思齐」的方法论：***
>
> ***目的***：*帮助团队既看到设计的精妙之处（理念好不好），也认清代码的真实质量（实现好不好）。避免把"理论上的好设计"等同于"实践中的好代码"。*
>
> ***方法***：*选取国际同类产品或知名开源项目作为参照系，每个评价维度分别给出"设计理念评级"和"工程实现评级"（1-5 星）。如果两者差异超过 1 星，明确说明差距原因。*
>
> ***典型结论***：*比如"流程编排模型设计理念 ★★★★☆，工程实现 ★★☆☆☆"——理念上三层配置+双层*

五、你们做什么会让我工作得更好

1. 写好需求定义

这是唯一完全由人类控制的输入，也是整条流水线的质量上限。

好的需求定义 = 好的 AI 输出

你写的内容	我能做到
明确的功能范围（In/Out Scope）	精确划定设计边界
可验证的验收标准（AC-01, AC-02...）	QA 能逐条核对
已知约束（必须兼容 XXX 表结构）	设计时主动规避
参考的现有实现（类似 XXX 接口）	快速定位变更模式

模糊的需求 = AI 猜测 = 返工

```
❌ "优化登录性能"              → 优化什么？目标是多少？我只能猜
✅ "登录接口 P99 从 500ms      → 明确的度量指标，我知道该做什么
   降到 200ms，主要瓶颈
   在密码校验环节"
```

2. 认真审核 Designer 的设计

这是你最重要的审核节点。 几个关键审查点：

检查 `[TASK_ALLOCATION]`

```
| 任务 | 修改文件 | 修改类型 |
|------|----------|----------|
| T-01 | FsLoginHandler.java | 修改 |   ← 这个文件改对了吗?
| T-02 | login_stat.sql | 新增 |         ← 需要新建 SQL 吗?
```

问自己：

- 该改的文件都列了吗?
- 有没有不该改的文件被列进来?
- 修改类型（新增/修改/删除）合理吗?

检查 `[DESIGN_DECISIONS]`

Designer 会记录他做的关键技术决策。如果你不同意某个决策，**现在改比 Coder 写完再改成本低 100 倍。**

3. 保持 project-context.md 准确

`project-context.md` 是我理解你项目的**唯一知识来源**。

如果里面写着 `Java 8`，我绝对不会用 `var` 关键字。
如果里面没说你有多数据源，我可能把 SQL 写到错误的数据库。

什么时候更新它?

变化	需要更新
新增了一个模块	✅ 更新 §3 模块职责
升级了 Spring Boot 版本	✅ 更新 §2 技术约束
改了构建命令	✅ 更新 §6 构建与测试
修了一个 bug	❌ 不需要
加了几个接口	❌ 不需要

> ***规则**：当你告诉我项目信息时（如"我们项目用了 Kafka"），*
> *我会写到 `project-context.md`，不会写到 `CLAUDE.md` 或 `AGENTS.md`。*

4. 善用 ilink-refine 处理不确定性

当我标记 `[待确认]` 时，**不要忽略它**。

```
❌ 直接 ilink-approve，跳过所有待确认项
✅ 执行 /ilink-refine，逐条和我确认决策
```

`ilink-refine` 的好处：

- 你的每个决策都被记录为 `[已确认 日期: 依据]`
- 下游角色会将 `[已确认]` 视为绑定约束，不再猜测
- 出了问题有据可查——是你确认的，还是我猜的，一目了然

5. 信任但验证

iLink 的设计哲学不是"完全信任 AI"，而是**"结构化信任"**：

```
PM 的输出     → 你可以不审（低风险，Designer 会审）
Designer 的输出 → 你必须审（⛔ Human-Gate）
Coder 的输出  → QA 先审，你做最终确认
QA 的输出     → 你只需看结论
```

你不需要审每一行代码。 你需要审的是：

1. 设计方案是否合理（Human-Gate）
2. QA 报告的结论是否可信（最终确认）

六、你们做什么会让我工作得更差

1. 跳过流水线步骤

```
❌ "不用做 PM 了，直接设计吧"
```

PM 阶段不是多余的。它的输出（业务合同）是 Designer 的输入契约。
跳过 PM = Designer 没有明确的业务边界 = 设计发散 = Coder 无所适从。

即使需求很简单，PM 阶段也能在 30 秒内完成。 但那 30 秒生成的业务合同，是后续所有角色的锚点。

2. 在 Coder 阶段追加需求

```
❌ "代码写得不错，顺便把 XXX 也加上吧"
```

Coder 有白名单约束。`[TASK_ALLOCATION]` 中没有的文件，我不应该碰。
如果你在 Coder 阶段追加需求，我要么：

- 违反白名单约束（破坏协议）
- 拒绝执行（你会觉得我不灵活）

正确做法：开一个新 Story，或者回到 Designer 阶段更新设计。

3. 修改 Master Doc 后不告诉我

```
❌ 手动编辑了 design.master.md，然后直接执行 /ilink-coder
```

iLink 的 Metadata 中有 `Upstream_SHA1`（上游文档哈希）。
如果你手动修改了 design.master.md 但没有更新 Metadata，我无法确认输入是否可信。

正确做法：如果你想修改设计，使用 `/ilink-refine` 与 Designer 角色修改，或手动修改后执行 `ilink-approve` 重新推进状态。

4. 把项目信息写到 CLAUDE.md / AGENTS.md

```
❌ 打开 CLAUDE.md，手动添加 "本项目使用 Java 8"
```

`CLAUDE.md` 和 `AGENTS.md` 是**薄路由文件**，只负责引导我去读 `project-context.md`。
如果你把项目信息写在两个地方，我会看到冲突的信息——`project-context.md` 说一个版本，`CLAUDE.md` 说另一个。

正确做法：所有项目信息写到 `project-context.md`。直接在对话中告诉我"请更新项目信息：XXX"，我会自动写到正确的文件。

5. 忽略 `[待确认]` 标记直接推进

```
❌ PM 输出了 STAGING（有待确认项），直接 ilink-approve 跳过
```

`[待确认]` 项会原样传入下游。如果 Designer 遇到一个 `[待确认]` 的业务规则，他只能基于假设做设计——假设错了，Coder 按假设写的代码也是错的。

正确做法：执行 `/ilink-refine` 逐条确认，或者至少有意识地承担不确定性风险。

七、常见误解澄清

误解 1："iLink 让 AI 替代开发者"

不是。 iLink 让 AI 承担重复性工作（分析、设计文档、编码、代码审查），
让开发者聚焦高价值工作（需求定义、设计决策、最终审核）。

你的角色从"写代码的人"变成"管 AI 写代码的人"。
你需要的技能不是减少了，而是升级了。

误解 2："AI 没有 iLink 也能写代码，iLink 是多余的"

确实，我没有 iLink 也能写代码。 但：

指标	无 iLink	有 iLink
上下文利用率	~25%（一个角色做所有事）	~100%（每个角色满血）
需求理解准确率	依赖你的描述完整度	有结构化需求定义
设计偏差风险	没有审查，我自己合理化	链式审查 + Human-Gate
不确定性处理	默默猜测	`[待确认]` 标记 + refine 确认
可追溯性	对话关了就没了	文件链永久保留
断点续跑	重新开始	读文件接上

类比：一个人也能盖房子，但有了施工规范、图纸审批、质量验收，房子更不容易塌。

误解 3："每个 Story 都要走完整流程，太慢了"

实际时间线：

```
PM 阶段          ← 30 秒 ~ 2 分钟
Designer 阶段    ← 1 ~ 3 分钟
Human-Gate       ← 取决于你审核的速度
Coder 阶段       ← 1 ~ 5 分钟
QA 阶段          ← 1 ~ 3 分钟
```

AI 执行的 4 个阶段加起来通常 < 10 分钟。 瓶颈不在 AI，在你审核设计的速度。

而这 10 分钟换来的是：完整的需求追踪、有据可查的设计决策、结构化代码审查。
如果你直接让我写代码，可能 3 分钟就写完了，但返工的概率和返工的成本都高得多。

误解 4："iLink 之后就不能直接和 AI 对话了"

完全可以。 iLink 是可选增强层，不是牢笼。

```
执行了 /ilink-bootstrap 之后:
├── 走 iLink 流水线 → 结构化协作
└── 直接对话 → 和之前完全一样
```

唯一的区别是：我读到 `CLAUDE.md` 中的 iLink 引导后，会知道项目信息在 `project-context.md`。
这让我即使在非 iLink 模式下也能更好地理解你的项目。

Bootstrap 本身就提升了 AI 的项目理解力，即使你从不执行 `/ilink-pm`。

误解 5："iLink 只能用一个 AI 工具"

iLink 是模型无关的。 同一个项目：

同事 A	同事 B	同事 C	同事 D
Claude Code	Codex CLI	Qoder	Gemini CLI
读 CLAUDE.md → 发现 iLink	读 AGENTS.md → 发现 iLink	读 .qoder/commands/ → 发现 iLink	读取 `.gemini/commands/` 目录 → 发现 iLink
读 project-context.md	读 project-context.md	读 project-context.md	读 project-context.md

所有人看到同一个 `project-context.md`，同一套 Soul 文件，同一份 Root Spec。

Story A 由 Claude 做 PM + Design，Story B 由 Qoder 做 Coder + QA，Story C 由 Gemini 做 Review——完全可以混用。

总结

从 AI 的视角，iLink 解决了三个问题

1. 我知道该读什么

```
以前：你不告诉我，我就不知道项目用 Java 8
现在：project-context.md 永远在那里，我每次都读
```

2. 我知道该做什么

```
以前：你说"改个登录接口"，我猜半天范围
现在：pm.master.md 告诉我业务边界
      design.master.md 告诉我改哪些文件
      我只需要精确执行
```

3. 我知道哪里会出错

```
以前：我写完代码自己看，看不出问题
现在：QA 角色（另一个 AI 实例）从零开始审查
      它不知道我写代码时的假设，所以能发现我的盲点
```

4. 我知道怎么处理不确定性

给你们的一句话

> ***iLink 不是让 AI 更强，而是让 AI 的弱点变得可控。***
>
> *我的弱点是：上下文有限、容易自我合理化、没有记忆、不擅长处理模糊性。*
> *iLink 的应对是：角色分工、链式审查、文件状态机、不确定性标记。*
>
> *你们的角色是：定义需求、审核设计、确认决策、最终把关。*
> *我们的角色是：分析、设计、编码、审查。*
>
> *这是一个互补的协作关系，不是替代关系。*

Q&A

问题?

附录：快速参考卡

命令	用途	AI 做什么
`/ilink-bootstrap`	项目冷启动	分析项目，生成 project-context.md
`/ilink-init <story>`	创建 Story	生成目录和需求模板
`/ilink-pm <story>`	需求分析	输出业务合同（边界 + 约束 + AC）
`/ilink-design <story>`	技术设计	输出设计方案 + 文件级任务清单
`ilink-approve <story>`	审核推进	（你来操作，不是 AI）
`/ilink-coder <story>`	编码	按设计直接写文件到磁盘
`/ilink-qa <story>`	代码审查	五步审查，输出结论
`/ilink-refine <story>`	修订 STAGING	逐条确认阻塞项，记录人类决策
`/ilink-domain <module>`	认知分析	阅读源码，生成 Domain Knowledge
`ilink-status [story]`	查看状态	显示流水线进度

项目知识库： `project-context.md`

根规范： `iLink/iLink-root-spec.md`